From Noise to Diversity: Random Embedding Injection in LLM Reasoning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Recent *soft prompt* methods, which prepend or insert trainable continuous embeddings into LLM inputs as virtual tokens, optimize these embeddings to enhance mathematical reasoning. However, the mechanism by which embedding injection itself affects model behavior, independent of optimization, remains unexplored. We investigate Random Soft Prompts (RSPs), continuous vectors sampled from an isotropic Gaussian fitted to the entrywise mean and variance of the pretrained embedding table and injected without any training. Theoretically, we show that the RSP contribution at each attention layer is exactly captured by a scalar random-token attention mass that is structurally diluted as autoregressive cache growth accumulates real tokens, and that under a local affine analysis of the perturbed logits RSPs can open previously excluded tokens into the effective top-K candidate set in a way temperature sampling provably cannot. Empirically, RSPs increase early-stage token entropy while generation stabilizes thereafter, and combining RSPs with temperature sampling yields higher trajectory diversity as measured by Pass@N. We further discuss the implications for GRPO training, where trajectory diversity is critical for learning signal quality.

1 Introduction

As large language models (LLMs) scale to billions of parameters, full fine-tuning becomes increasingly expensive. This has motivated parameter-efficient methods that adapt frozen models by introducing a small number of trainable parameters [Hu et al., 2022, Houlsby et al., 2019]. Among these, *soft prompts* prepend or insert learnable continuous embeddings into the input sequence, steering model behavior through the attention mechanism [Li and Liang, 2021, Lester et al., 2021]. This paradigm has been actively adopted for enhancing mathematical reasoning in LLMs [Kang et al., 2026, Xu et al., 2025, Ye et al., 2026, Hao et al., 2025, Zhang et al., 2026]. Despite their diverse designs, these methods share a common structure: they inject continuous embeddings that lie outside the pretrained vocabulary and optimize them to improve downstream performance.

However, we find that optimization is not a necessary condition for these effects. We observe that injecting randomly initialized, untrained continuous embeddings can also shift the output distribution. For instance, applying a chat template to a base model not fine-tuned with such format causes a prompt-format mismatch that degrades reasoning capability, but appending random embeddings to such mismatched inputs mitigates this degradation. If simple noise were merely perturbing the model, accuracy should worsen instead. This suggests that injecting out-of-vocabulary continuous embeddings itself, independent of any learned content, plays a non-trivial role in shaping model behavior. Existing studies primarily evaluate *soft prompt* methods through end-task accuracy, and the mechanisms by which embedding injection influences the output distribution remain insufficiently explored.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

In this work, we investigate the role of Random Soft Prompts (RSPs), continuous vectors sampled from an isotropic Gaussian fitted to the entrywise mean and variance of the pretrained embedding table and injected without any training. The role of randomness is not to imitate the embedding distribution; it is to ensure that repeated rollouts receive *independent* perturbations, which is what exploration requires. Theoretically, we identify an attention-level mechanism in which the RSP contribution is exactly captured by a scalar random-token attention mass that is structurally diluted by autoregressive cache growth, and we show that under a local affine analysis of the perturbed logits RSPs can open previously excluded tokens into the effective top-K candidate set in a way temperature sampling provably cannot. Empirically, RSPs increase early-stage token entropy while the output stabilizes in later generation stages, and combining RSPs with temperature sampling yields more diverse reasoning trajectories as measured by Pass@N scaling. We further discuss the implications of these findings for Group Relative Policy Optimization (GRPO) [Shao et al., 2024], where trajectory diversity is essential for effective learning signals.

Our contributions are as follows:

- We give a transformer-level mechanism in which the RSP contribution is controlled by a scalar attention mass with structural cache-driven dilution, and a distributional analysis under a local affine surrogate showing that *independent isotropic resampling*—not a fixed perturbation—can admit previously excluded tokens into the effective top-K candidate set in a way temperature sampling cannot.
- We empirically analyze how RSPs affect LLM generation, showing that RSPs increase early token entropy and produce more diverse reasoning trajectories when combined with temperature sampling, and that the accumulation gain disappears when one RSP is shared across samples.
- We discuss the implications of RSP-induced diversity for GRPO training and argue that RSP, being rank-changing while temperature is rank-preserving, provides exploration gains complementary to existing temperature-based approaches.

2 Background

2.1 Soft Prompt Methods for LLM Reasoning

Soft prompts emerged as a parameter-efficient alternative to full fine-tuning. Prefix-Tuning [Li and Liang, 2021] introduced the paradigm by prepending trainable continuous vectors to every transformer layer, Prompt Tuning [Lester et al., 2021] simplified this to input-layer-only tuning and showed that it matches full fine-tuning at sufficient scale, and P-tuning [Liu et al., 2024] extended the approach with an LSTM-based prompt encoder to model inter-position dependencies.

More recently, *soft prompts* have been actively applied to LLM reasoning. TTSV [Kang et al., 2026] optimizes prefix embeddings at test time via entropy minimization, steering the model toward high-certainty states to activate latent reasoning capabilities. SoftCoT [Xu et al., 2025] replaces explicit chain-of-thought tokens with soft tokens generated by an assistant model, leveraging the stronger representational capacity of continuous representations over discrete language. LTPO [Ye et al., 2026] optimizes latent thought embeddings per test instance through policy gradient with a confidence-based intrinsic reward. COCONUT [Hao et al., 2025] feeds the model's own hidden states back as input embeddings, enabling reasoning in a continuous latent space. MemGen [Zhang et al., 2026] demonstrates that embedding vectors can serve as experience memory by constructing latent token sequences that enrich the model's reasoning. Pause tokens [Goyal et al., 2024] insert learnable tokens into the input to provide additional computation steps.

These approaches all inject out-of-vocabulary continuous embeddings optimized for task-specific objectives. Because evaluation centers on end-task accuracy, the contribution of the injection itself and that of the optimization are not separated, and the mechanisms by which such out-of-vocabulary continuous embeddings shape model behavior remain a separate and underexplored question.

2.2 Noise Injection in Neural Networks

Neural networks have incorporated noise at various stages. During training, dropout [Srivastava et al., 2014] randomly deactivates hidden units to prevent overfitting, and NEFTune [Jain et al., 2024] adds noise to token embeddings during instruction fine-tuning, reporting improvements in instruction following. These methods use noise as a regularizer and do not apply perturbations at inference time.

At inference time, perturbation-based approaches primarily target robustness and safety. Randomized smoothing [Cohen et al., 2019] injects Gaussian noise into inputs to obtain certified robustness guarantees. In the LLM setting, SmoothLLM [Robey et al., 2025] perturbs input prompts at the character level to defend against adversarial jailbreaking. These methods require multiple noisy forward passes with output aggregation, and target prediction stability rather than output diversity.

In reinforcement learning, NoisyNet [Fortunato et al., 2018] injects learnable noise into network weights to improve exploration, demonstrating that the location and structure of noise injection affect exploration quality. This motivation is closest to RSPs, which similarly target diversity, but differ in injection target (input embeddings vs. network weights) and require no learned noise parameters.

RSPs differ from these approaches in the following respects. **(1)** RSPs operate solely at inference without any training, whereas dropout and NEFTune inject noise during training, modifying the resulting model parameters accordingly. **(2)** RSPs preserve the original input and append fresh embedding vectors in a single forward pass, whereas robustness methods corrupt the input and aggregate over multiple noisy passes. **(3)** RSPs use purely random vectors sampled from the pretrained embedding statistics, whereas NoisyNet learns noise parameters through optimization.

3 Random Soft Prompts and Why They Work

This section states the mechanism used in the paper. A *rollout* means one full autoregressive decode under one sampled RSP. Within a rollout, an RSP enters the hidden state only through attention; cache growth structurally dilutes its contribution, producing an automatic explore-then-exploit annealing without any explicit schedule. Across rollouts, independent resampling can accumulate early branch differences into Pass@N gains. We keep the main statements and intuition here; the full derivations are in Appendix D.

3.1 Random Soft Prompt

We start with the object being analyzed. Let $\mathbf{W}_E \in \mathbb{R}^{V\times d}$ denote the pretrained token-embedding matrix, where V is the vocabulary size and d is the hidden dimension. Write its entrywise mean and standard deviation as $\mu_E := \frac{1}{Vd}\sum_{i,k}[\mathbf{W}_E]_{i,k}$ and $\sigma_E := \mathrm{std}(\mathbf{W}_E)$. An RSP of length L is a sequence of continuous vectors $\mathbf{H} = [h_1, \dots, h_L] \in \mathbb{R}^{L\times d}$ drawn independently from

$$h_j \;\sim\; \mathcal{N}(\mu_E \mathbf{1}_d,\; \sigma_E^2 \mathbf{I}_d), \quad j = 1, \dots, L. \tag{1}$$

The centered form $\bar{h}_j := h_j - \mu_E \mathbf{1}_d \sim \mathcal{N}(0, \sigma_E^2 \mathbf{I}_d)$ is a zero-mean isotropic Gaussian. The prompt can be injected as a prefix $[\mathbf{H}; \mathbf{X}]$, infix, or suffix $[\mathbf{X}; \mathbf{H}]$ [Kang et al., 2026, Xu et al., 2025, Ye et al., 2026, Zhang et al., 2026]. For repeated-sampling protocols, prompt draws and any decoding randomness across compared trials are assumed independent.

The role of randomness is not to imitate the empirical embedding distribution. It is to make repeated rollouts from the same text prompt receive *independent* perturbations. That independence is what exploration requires.

3.2 Attention decomposition and automatic annealing

We first ask how a single RSP injection perturbs the hidden state. RSP does not add to the hidden state directly; it enters only through attention weights. So its entire instantaneous influence at one layer can collect into one quantity: the attention mass assigned to RSP tokens.

Concretely, consider self-attention layer ℓ with n real tokens and L RSP tokens, attention weights $\alpha_{ij}^{(\ell)}$, and value vectors $\mathbf{v}_j^{(\ell)}, \mathbf{v}_{r_j}^{(\ell)}$ on the real and RSP sides. Define the RSP attention mass $w_{r,i}^{(\ell)} := \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)}$. The attention output decomposes *exactly* as

$$\mathbf{x}_i^{(\ell+1)} \;=\; (1 - w_{r,i}^{(\ell)})\,\tilde{\mathbf{x}}_i^{(\ell+1)} \;+\; \boldsymbol{\eta}_i^{(\ell)}, \quad \tilde{\mathbf{x}}_i^{(\ell+1)} := \sum_{j=1}^{n} \frac{\alpha_{ij}^{(\ell)}}{1-w_{r,i}^{(\ell)}}\,\mathbf{v}_j^{(\ell)}, \quad \boldsymbol{\eta}_i^{(\ell)} := \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)} \mathbf{v}_{r_j}^{(\ell)}. \tag{2}$$

Derivation of Eq. (2). Split the attention output $\sum_{j=1}^{n+L} \alpha_{ij}^{(\ell)} \mathbf{v}_{ij}^{(\ell)}$ into the real-token sum and the RSP-token sum, and factor $1 - w_{r,i}^{(\ell)} = \sum_{j=1}^{n} \alpha_{ij}^{(\ell)}$ out of the real part. No approximation beyond the softmax-normalization identity is used. $\square$

The same scalar $w_{r,i}^{(\ell)}$ controls two things at once. The attenuation ratio $1 - w_{r,i}^{(\ell)}$ on the real-token signal and the upper bound $\|\boldsymbol{\eta}_i^{(\ell)}\| \le w_{r,i}^{(\ell)} \max_j \|\mathbf{v}_{r_j}^{(\ell)}\|$ on the random contribution are both tied to it. The RSP-induced variation reduces to tracking a single number in $[0, 1]$. The natural next question: how does this number behave as decoding proceeds? In autoregressive generation, the KV cache grows by one token each step, the softmax denominator grows with it, and so the RSP share should naturally shrink. The intuition becomes an inequality.

Theorem 1 (Attention-mass decay under cache growth)**.** *Let* $\Delta_i^{(\ell)} := \min_{j \in [n]} \mathbf{q}_i^\top \mathbf{k}_j - \max_{j' \in [L]} \mathbf{q}_i^\top \mathbf{k}_{r_{j'}}$ *denote the pre-scale logit gap between real and random tokens. In scaled dot-product attention,*

$$w_{r,i}^{(\ell)} \le \frac{L}{L + n \exp(\Delta_i^{(\ell)}/\sqrt{d})},$$

the right-hand side is strictly decreasing in n at fixed L and $\Delta_i^{(\ell)}$, and $w_{r,i}^{(\ell)} \to 0$ as $n \to \infty$.

Proof. Write $s_j := \mathbf{q}_i^\top \mathbf{k}_j/\sqrt{d}$, $s_{r_{j'}} := \mathbf{q}_i^\top \mathbf{k}_{r_{j'}}/\sqrt{d}$, and $s_{r,\max} := \max_{j'} s_{r_{j'}}$. The gap definition gives $s_j \ge \Delta_i^{(\ell)}/\sqrt{d} + s_{r,\max}$ for every real token j. Let $R := \sum_{j'=1}^{L} \exp(s_{r_{j'}})$; since $\exp(s_{r,\max}) \ge R/L$, summing over the n real keys yields $\sum_{j=1}^{n} \exp(s_j) \ge n \exp(\Delta_i^{(\ell)}/\sqrt{d}) \cdot R/L$. Substituting into the softmax-normalized RSP mass, $w_{r,i}^{(\ell)} = R/(\sum_j \exp(s_j) + R) \le L/(L + n \exp(\Delta_i^{(\ell)}/\sqrt{d}))$. Differentiating the right-hand side with respect to n gives a strictly negative value, so the bound is strictly decreasing, and it tends to 0 as $n \to \infty$. $\square$

This single inequality carries two messages. The RSP length L is the design parameter we choose; it sits in the numerator and sets the *maximum* influence of the perturbation. The real-token count n grows automatically every step and sits in the denominator, so the *envelope* of the influence shrinks under the same L. What the theorem guarantees is the sequence-direction attenuation of the upper envelope at fixed (layer, query position, logit gap), not a layer-direction monotone decay; queries, keys, and hidden states co-evolve, so $\Delta_i^{(\ell)}$ also moves and the realized $w_{r,i}^{(\ell)}$ need not decrease at every step. The accurate reading is that "the envelope compatible with a fixed gap shrinks as n grows." The full transformer block adds residuals, normalization, and an FFN, but under a common local Lipschitz constant on a ball containing both perturbed and unperturbed trajectories [Xiong et al., 2020, Kim et al., 2021], Appendix D shows that the per-block perturbation amplitude is linear in $w_{r,i}^{(\ell)}$ and a finite-depth cumulative bound follows. Combined with Theorem 1, the perturbation is large in early blocks and weakens as evidence accumulates—no chain-long persistence is required. Figure 1 verifies the sequence-direction prediction empirically.

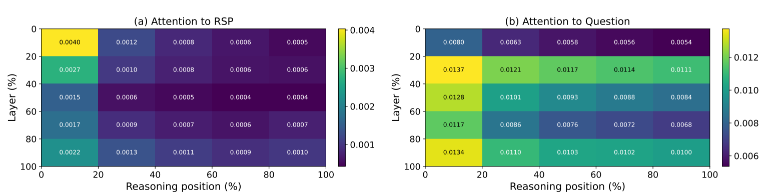


Figure 1: Per-token attention mass on Qwen2.5-Math-7B (suffix, 500 MATH-500 problems, each with an independently sampled RSP). (a) RSP-token attention decreases along the reasoning-position axis, consistent with the cache-growth bound of Theorem 1. The additional decrease along the layer axis is a separate empirical phenomenon suggesting that deeper layers sharpen the real-vs-random logit gap; it does not follow from Theorem 1 alone. (b) Question-token attention remains comparatively uniform. The reported quantity is the per-token mass of Appendix E, equal to $w_{r,i}^{(\ell)}/L$ averaged over heads and samples.

3.3 Minimax justification of the isotropic Gaussian

§3.2 explains *how* the RSP enters the attention output, but it does not justify the choice of distribution. Natural alternatives—a learned fixed prompt, random vocabulary tokens, or noise matched to the embedding covariance—are all on the table.

The decisive direction that opens a branch differs from problem to problem and is not known in advance, so any distribution that concentrates variance along a few directions under-explores the others. To explore every direction as evenly as possible under a fixed variance budget, one wants a distribution that distributes that budget across all directions without bias. A minimax statement formalizes this requirement.

Theorem 2 (Direction-agnostic minimax coverage)**.** *Let D be any zero-mean distribution on $\mathbb{R}^d$ with covariance Σ_D and trace budget $\operatorname{tr}(\Sigma_D) \le \rho^2$. Then $\min_{\|b\|=1} \operatorname{Var}_{h\sim D}(b^\top h) = \lambda_{\min}(\Sigma_D) \le \rho^2/d$, with equality if and only if $\Sigma_D = (\rho^2/d)\mathbf{I}_d$.*

Equality holds for any distribution with isotropic covariance—Gaussianity is not the unique answer. A learned fixed prompt has $\Sigma_D = 0$ and explores no direction at all; PCA-aligned noise leans on a few directions and under-explores the orthogonal complement; vocabulary-token sampling adds a directional bias on top of pretrained semantic priors; and even "use the empirical embedding covariance as is" is anisotropic and so under-explores at least one direction. The Gaussian RSP design *borrows the scale from the embedding distribution but discards its directional geometry on purpose*. The isotropic Gaussian additionally has positive density on all of $\mathbb{R}^d$ (full support), so it places no prior weight on any direction (Appendix C).

3.4 Output support expansion and Pass@N accumulation

Combining the two preceding results pins down what *one* RSP can do to the output distribution—and what it cannot. Together, the nonzero attention-side influence (§3.2) and the direction-agnostic positive-probability prompt law (§3.3) imply that, on some prompt draw, a token barely visible under the baseline can enter the local top-K candidate set—provided such a region exists on the surrogate. The mechanism assigns positive probability to that region; whether the region is nonempty is a task-side condition.

Even when the single-draw probability is small, *independent* resampling accumulates it quickly. The probability of entering at least once across N fresh RSP draws approaches 1 at an exponential rate. Temperature sampling, which is a monotone transform of the logits, can never reach this region because it cannot change the token ranking.

Linearizing the logits around the centered prompt $\bar{h}$ gives the surrogate $z_i^{\mathrm{lin}}(\bar{h}) := z_i + a_i^\top \bar{h}$ with $a_i := \nabla_{\bar{h}} z_i(0)$. The top-$K$ entry region for a baseline-excluded token i is $\mathcal{G}_{i,K} := \big\{\bar{h} \in \mathbb{R}^d : \#\{j \neq i : z_j^{\mathrm{lin}}(\bar{h}) > z_i^{\mathrm{lin}}(\bar{h})\} \le K-1\big\}$, matching the actual top-K definition.

Proposition 1 (Conditional top-K entry)**.** *If $\mathcal{G}_{i,K}$ contains a nonempty open set, the isotropic Gaussian's full support assigns it positive probability, so $\mu_i := \Pr_{\bar{h}}(\bar{h} \in \mathcal{G}_{i,K}) > 0$.*

The mechanism guarantees that *branches can open* ($\mu_i > 0$); the existence of such a region is a task-side condition on the surrogate that is not automatic. To lift this single-draw probability to task accuracy, we add two task-side assumptions: (M1) each independent RSP run reaches a correct trajectory with marginal probability at least $p_{\min}(x) > 0$, and (M2) the N runs are mutually independent.

Proposition 2 (Multi-path Pass@N)**.** *Under (M1)–(M2), with N independent RSP runs, $\Pr\big(\text{Pass@}N \text{ on } x\big) \;\ge\; 1 - (1 - p_{\min}(x))^N \;\ge\; 1 - \exp(-N\, p_{\min}(x))$.*

This division of labor is the main interpretive point. The mechanism is responsible for *admitting* baseline-excluded candidates into the local candidate set; whether an admitted branch is *productive* is task-dependent, which is why $p_{\min}(x) > 0$ is a separate assumption. Sharing one RSP across all samples removes this accumulation, so *independence* is essential. Section 4 tests exactly this signature.

Table 1: Accuracy (%) across model configurations and benchmarks. RSP reports the best result across injection positions (Prefix, Infix, Suffix). Full per-position breakdown is in Appendix A. Values in parentheses indicate the difference from the baseline.

	MATH-500		**GSM8K**		**AIME24**	
Model	Baseline	RSP	Baseline	RSP	Baseline	RSP
Instruct Models						
LLaMA-3.1-8B-Inst	52.20	49.40 (−2.8)	85.75	86.73 (+1.0)	6.67	10.00 (+3.3)
Qwen2.5-Math-7B-Inst	83.20	84.20 (+1.0)	95.45	95.68 (+0.2)	13.33	16.67 (+3.3)
Qwen2.5-Math-1.5B-Inst	73.00	75.20 (+2.2)	85.29	85.75 (+0.5)	10.00	20.00 (+10.0)
Base Models (Raw Text)						
Qwen2.5-Math-7B	72.20	71.60 (−0.6)	84.15	84.31 (+0.2)	6.67	16.67 (+10.0)
Qwen2.5-Math-1.5B	61.40	64.40 (+3.0)	77.86	74.30 (−3.6)	16.67	10.00 (−6.7)
Base Models + ChatML (Format Mismatch)						
Qwen2.5-Math-7B	52.20	70.40 (+18.2)	58.30	75.97 (+17.7)	23.33	23.33 (+0.0)
Qwen2.5-Math-1.5B	34.40	63.40 (+29.0)	37.45	67.02 (+29.6)	13.33	20.00 (+6.7)

4 Empirical Validation

We now examine how the explanation in §3 appears in actual LLM generation through three scenes. First, we ask whether RSP preserves baseline accuracy without breaking it, and recovers it in some settings (§4.2). Second, we check whether the early diversification followed by late stabilization—predicted by the two-sided envelope—is observed empirically (§4.3). Finally, we contrast Pass@N between sharing one RSP across samples versus drawing a fresh RSP per sample, directly probing whether *independence* is the key (§4.4).

4.1 Experimental Setup

We evaluate on three mathematical reasoning benchmarks, MATH-500 [Lightman et al., 2024], GSM8K [Cobbe et al., 2021], and AIME24, using LLaMA-3.1-8B-Instruct [Grattafiori et al., 2024] and the Qwen2.5-Math model family [Yang et al., 2024] across instruct models, base models, and base models with mismatched chat templates—seven configurations in total. All experiments use greedy decoding to isolate the effect of RSPs from sampling stochasticity, and the answer-extraction pipeline uses fallbacks to score only the final answer. RSPs follow the definition of §3.1 and are concatenated at one of three positions (prefix, infix, suffix) with a freshly drawn $\mathbf{H}$ for each batch. Full experimental details and per-position results are in Appendix A.

4.2 How Does Untrained RSP Compare to Baselines & Optimized Soft Prompts?

We first examine how injecting random embeddings affects the model's reasoning performance. Table 1 reports the accuracy for each model configuration, using the best RSP result across injection positions (Prefix, Infix, Suffix) and $L \in \{10, 15, 20\}$ for each benchmark.

For instruct models and base models with their native prompt formats, RSP injection maintains or slightly shifts baseline performance within a few percentage points. Most strikingly, under prompt-format mismatch — where base models receive ChatML templates they were not trained on — RSP yields pronounced improvements of up to $+29$ pp. These gains are difficult to attribute to simple perturbation effects, as noise that merely raises model uncertainty would further degrade performance.

We compare RSPs with prior soft prompt methods (TTSV, SoftCoT, LTPO) that optimize their embeddings for distinct objectives, under a unified evaluation protocol reported in Table 2. All methods are re-evaluated with the same unified extraction pipeline (Appendix A.2), enabling a format-agnostic comparison despite each method's distinct prompt format and grading scheme. The results place RSPs on par with the optimized methods without requiring any training.

Table 2: Comparison with existing soft prompt methods (TTSV, SoftCoT, LTPO) under unified evaluation. Baseline and RSP values are from Table 1. **Bold** denotes the best result and underline denotes the second best for each model–benchmark pair.

Model	Benchmark	Baseline	RSP	TTSV	SoftCoT	LTPO
Qwen2.5-Math-7B-Instruct	MATH-500	83.20	**84.20**	83.80	82.80	83.00
	GSM8K	95.45	**95.68**	95.30	95.00	94.84
	AIME24	13.33	16.67	**23.30**	16.67	13.33
Qwen2.5-Math-1.5B-Instruct	MATH-500	73.00	75.20	75.20	**76.00**	72.40
	GSM8K	85.29	**85.75**	85.70	84.46	84.53
	AIME24	10.00	**20.00**	6.70	6.67	10.00

4.3 How Does RSP Affect the Output Distribution?

We analyze how RSP injection changes the model's output distribution on Qwen2.5-Math-1.5B-Instruct with MATH-500, measuring per-token entropy, top-1 probability, and varentropy across the generation process. Figure 2 compares these metrics for the first 5% of generation steps across RSP variants and prior soft prompt methods (LTPO, SoftCoT, TTSV). Full statistics are in Appendix A.4.

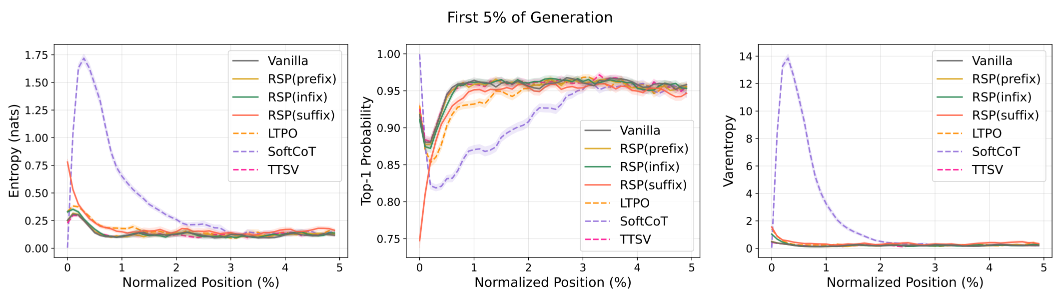


Figure 2: Mean entropy, top-1 probability, and varentropy during the first 5% of generation steps (Qwen2.5-Math-1.5B-Instruct, MATH-500). Shading indicates ± 1 SEM. Solid lines: RSP variants and the baseline. Dashed lines: prior soft prompt methods (LTPO, SoftCoT, TTSV).

These results show that RSP modifies the early-stage output distribution across all injection positions: entropy increases (the token distribution flattens), top-1 probability decreases (commitment to a dominant token weakens), and varentropy increases (indicating multimodal distributions [Ahmed et al., 2026]). High-entropy positions are known to act as critical forking points that determine reasoning trajectories [Wang et al., 2025], so this shift moves the model toward a state where multiple reasoning paths can coexist. The change is localized to the initial reasoning stage. In the middle and final portions of generation, all three metrics converge to baseline levels.

Although existing soft prompt methods are optimized for different objectives, they share a common pattern of increasing early-stage entropy. Notably, TTSV, which optimizes soft prompts using a reward that reduces the mean entropy of reasoning trajectories, still exhibits early entropy comparable to the baseline. This suggests that elevating early-stage entropy is an inherent effect of injecting non-pretrained continuous embeddings, independent of the optimization objective.

4.4 Does RSP Induce Trajectory Diversity?

Section 4.3 empirically established that RSP reshapes the early-stage output distribution. We now examine whether this shift translates into reasoning diversity at three complementary levels of analysis: token rankings, semantic content of trajectories, and task-level outcomes. The three analyses converge on a directional picture: RSP's first-token perturbation propagates into semantically more diverse reasoning trajectories, which in turn yields greater output diversity at the task level. All three analyses share the setting: Qwen2.5-Math-1.5B-Instruct, MATH-500, suffix injection, and $L = 10$.

Token-level: distribution beyond temperature. We test whether RSP modifies the first-token distribution

Table 3: First-token distribution metrics measured against the baseline at τ=1.0, comparing the baseline at τ=2.0 with RSP at τ=1.0.

Metric	τ=2.0	RSP
Spearman ρ	1.0	**0.651**
Mass outside top-10	5.15%	**27.64%**
JS divergence	0.086	**0.231**

in a qualitatively different way from temperature scaling, which preserves token rankings while only flattening probabilities. Under autoregressive decoding, any trajectory-level divergence between RSP and the baseline must originate from the first generation step, a critical forking point for reasoning trajectories [Wang et al., 2025]. Using the baseline at τ=1.0 as reference, we compare RSP at τ=1.0 against the baseline at τ=2.0 (the high-temperature ceiling of what flattening alone can achieve) on three complementary metrics: Spearman rank correlation ρ to detect rank reorganization, the probability mass placed outside the baseline's top-10 to measure support expansion, and Jensen–Shannon divergence to quantify the overall magnitude of distributional change (formal definitions in Appendix B).

Table 3 reports the contrast. Even at τ=2.0, only 5.15% of probability mass falls outside the baseline's top-10. RSP at τ=1.0 reaches 27.64%. The Spearman rank correlation tells the story: because temperature scaling $p_i^{(\tau)} \propto p_i^{1/\tau}$ is a monotone transform that preserves token ranking ($p_i > p_j \iff p_i^{(\tau)} > p_j^{(\tau)}$) for any $\tau > 0$, ρ relative to the baseline is exactly 1. RSP, by contrast, reranks the token distribution itself, reducing ρ to 0.651—an effect temperature cannot produce at any τ. The Jensen–Shannon divergence (0.231) quantifies the overall magnitude of this distributional change. Since $\rho < 1$ rules out rank-preserving flattening as the sole cause, the high divergence reflects a structural reshape of the distribution rather than a uniform flattening of the same ranking.

Trajectory-level: semantic diversity. Token-level reranking raises a second question: do first-token perturbations produce semantically diverse reasoning trajectories, or do independently perturbed trajectories converge to semantically similar ones? We randomly sample 64 problems from MATH-500 and generate 300 independent trajectories per problem under Baseline and RSP at temperature $\tau = 1.0$. Each trajectory is encoded into a dense semantic vector by BGE-M3 [Chen et al., 2024], a sentence-level multilingual embedding model, and we compute three complementary metrics over these embeddings: *pairwise cosine distance* between trajectories captures the overall semantic spread of the trajectory set; *inter-cluster distance* between centroids identified by DBSCAN [Ester et al., 1996], a density-based clustering algorithm that groups nearby trajectories without prespecifying the number of clusters, captures the separation between distinct trajectory groups; and *intra-cluster distance* captures semantic coherence within each group.

Table 4: Semantic diversity metrics (64 problems, 300 trajectories per condition).

Metric	Baseline	RSP
Pairwise cos. dist.	0.0612	**0.0879**
Inter-cluster dist.	0.0183	**0.0982**
Intra-cluster dist.	0.0526	0.0528

Table 4 reveals three concurrent patterns. Pairwise cosine distance rises by about $1.4\times$, indicating that the trajectory set as a whole becomes more semantically diverse. Inter-cluster distance jumps by about $5.4\times$, showing that the trajectories split into substantially more separated groups. Intra-cluster distance is essentially unchanged, indicating that semantic coherence within each group remains comparable to the baseline. RSP also yields larger pairwise distance than baseline on 56 of 64 problems (87.5%).

Outcome-level: Pass@n scaling. Finally, we evaluate whether the token- and trajectory-level diversity translates into task-level outcome diversity through Pass@n [Chen et al., 2021], the probability that at least one of n sampled solutions is correct. We compare four conditions with $n = 16$ samples per problem on MATH-500 and AIME24: *(i) Baseline,* $\tau = 1.0$: temperature sampling only; *(ii) RSP (fixed seed),* $\tau = 1.0$: a single RSP shared across all samples; *(iii) RSP (indep. seed), greedy*: a different RSP per sample without temperature; and *(iv) RSP (indep. seed),* $\tau = 1.0$: a different RSP per sample combined with temperature sampling.

Condition *(iv)* achieves the highest Pass@n on both benchmarks, reaching 92.80% on MATH-500 and 36.67% on AIME24 at $n = 16$. Condition *(ii)* shows no improvement over the baseline, suggesting that the diversity gain likely depends on varying the random embeddings across samples rather than a single fixed perturbation. Pass@1 remains comparable to the baseline on MATH-500, and on the more challenging AIME24, *(iv)* achieves higher Pass@1 than the baseline. Together these observations suggest that combining independent RSPs with temperature sampling may yield greater trajectory diversity while preserving single-sample accuracy.

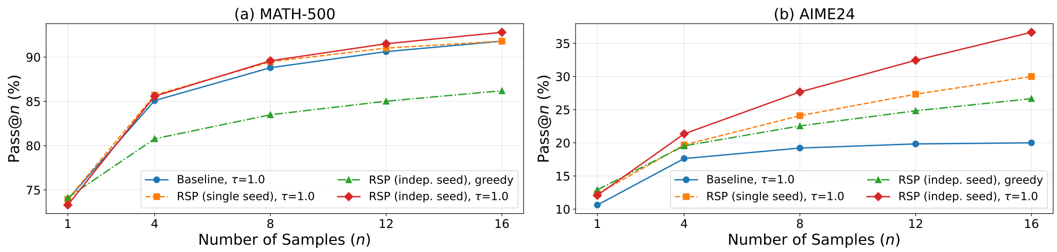


Figure 3: Pass@n scaling on (a) MATH-500 and (b) AIME24 with Qwen2.5-Math-1.5B-Instruct, $n = 16$ samples per problem. Baseline: temperature sampling only. Fixed seed: single RSP shared across samples combined with temperature. Indep. seed: a different RSP per sample, with or without temperature.

5 Conclusion

We studied Random Soft Prompts (RSPs), isotropic Gaussian vectors fitted to the entrywise mean and variance of the pretrained embedding table and injected without training. Theoretically, the RSP contribution at each attention layer is exactly captured by a scalar attention mass that cache growth structurally dilutes, and under a local affine analysis RSPs can admit previously excluded tokens into the effective top-K candidate set in a way temperature scaling cannot (§3). Empirically, RSPs raise early-stage entropy while later generation stabilizes, and the accumulation gain disappears when one RSP is shared across samples (§4). Because RSP is rank-changing while temperature is rank-preserving, the two are complementary, suggesting a sampling-level remedy for entropy collapse in GRPO [Shao et al., 2024] beyond DAPO [Yu et al., 2025] and CE-GPPO [Su et al., 2026]; training-time validation, extension beyond mathematical reasoning, and a principled choice of RSP length and injection position—together with the task-side bound $p_{\min}(x){>}0$ of Proposition 2—are natural next steps.

References

Farhan Ahmed, Yuya Jeremy Ong, and Chad DeLuca. LogitScope: A Framework for Analyzing LLM Uncertainty through Information Metrics. In *ICLR Workshop*, 2026.

Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting Latent Predictions from Transformers with the Tuned Lens. *arXiv:2303.08112*, 2023.

Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of ACL*, 2024.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*, 2021.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168*, 2021.

Jeremy M. Cohen, Elan Rosenfeld, and J. Zico Kolter. Certified Adversarial Robustness via Randomized Smoothing. In *ICML*, 2019.

Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. In *KDD*, 1996.

Meire Fortunato, Mohammad Gheshlaghi Azar, Bilal Piot, Jacob Menick, Ian Osband, Alex Graves, Vlad Mnih, Remi Munos, Demis Hassabis, Olivier Pietquin, Charles Blundell, and Shane Legg. Noisy Networks for Exploration. In *ICLR*, 2018.

Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer Feed-Forward Layers Are Key-Value Memories. In *EMNLP*, 2021.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before You Speak: Training Language Models with Pause Tokens. In *ICLR*, 2024.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vítor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban

Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. The Llama 3 Herd of Models. *arXiv:2407.21783*, 2024.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training Large Language Models to Reason in a Continuous Latent Space. In *ICLR*, 2025.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. In *ICML*, 2019.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*, 2022.

Neel Jain, Ping-yeh Chiang, Yuxin Wen, John Kirchenbauer, Hong-Min Chu, Gowthami Somepalli, Brian R. Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Aniruddha Saha, Micah Goldblum, Jonas Geiping, and Tom Goldstein. NEFTune: Noisy Embeddings Improve Instruction Finetuning. In *ICLR*, 2024.

Xinyue Kang, Diwei Shi, and Li Chen. Model Whisper: Steering Vectors Unlock Large Language Models' Potential in Test-time. In *AAAI*, 2026.

Hyunjik Kim, George Papamakarios, and Andriy Mnih. The Lipschitz Constant of Self-Attention. *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.

Brian Lester, Rami Al-Rfou, and Noah Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. In *EMNLP*, 2021.

Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *ACL-IJCNLP*, 2021.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's Verify Step by Step. In *ICLR*, 2024.

Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. GPT Understands, Too. *AI Open*, 5:208–215, 2024.

Aman Madaan and Amir Yazdanbakhsh. Text and Patterns: For Effective Chain of Thought, It Takes Two to Tango. *arXiv:2209.07686*, 2022.

Alexander Robey, Eric Wong, Hamed Hassani, and George J. Pappas. SmoothLLM: Defending Large Language Models Against Jailbreaking Attacks. *Transactions on Machine Learning Research*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*, 2024.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.

Zhenpeng Su, Leiyu Pan, Minxuan Lv, Yuntao Li, Wenping Hu, Fuzheng Zhang, Kun Gai, and Guorui Zhou. CE-GPPO: Coordinating Entropy via Gradient-Preserving Clipping Policy Optimization in Reinforcement Learning. In *ACL*, 2026.

Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 Rule: High-Entropy Minority Tokens Drive Effective Reinforcement Learning for LLM Reasoning. In *NeurIPS*, 2025.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On Layer Normalization in the Transformer Architecture. In *International Conference on Machine Learning (ICML)*, 2020.

Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. SoftCoT: Soft Chain-of-Thought for Efficient Reasoning with LLMs. In *ACL*, 2025.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. *arXiv:2409.12122*, 2024.

Wengao Ye, Yan Liang, and Lianlei Shan. Thinking on the Fly: Test-Time Reasoning Enhancement via Latent Thought Policy Optimization. In *ICLR*, 2026.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Juncai Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Ru Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Yonghui Wu, and Mingxuan Wang. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. In *NeurIPS*, 2025.

Guibin Zhang, Muxin Fu, and Shuicheng Yan. MemGen: Weaving Generative Latent Memory for Self-Evolving Agents. In *ICLR*, 2026.

A Experimental Setup and Full Accuracy Breakdown

Table 5 provides the full per-position accuracy breakdown (Prefix, Infix, Suffix) corresponding to the best-across-positions results summarized in Table 1 in the main text. Table 6 lists the number of RSP tokens L used for each model–benchmark pair, selected from $\{10, 15, 20\}$. All experiments are conducted on a single NVIDIA A6000 40GB GPU with greedy decoding, a maximum generation length of 3,072 tokens for MATH-500 and GSM8K, and 4,096 tokens for AIME24. Batch size is fixed per model (16 for LLaMA-3.1-8B-Instruct and Qwen2.5-Math-7B variants, 32 for Qwen2.5-Math-1.5B variants).

Table 5: Accuracy (%) across model configurations, injection positions, and benchmarks. Values in parentheses indicate the difference from the baseline. **Bold** denotes the best result and underline denotes the second best for each model–benchmark pair.

Model	Mode	MATH-500	GSM8K	AIME24
Instruct Models				
LLaMA-3.1-8B-Instruct	Baseline	**52.20**	85.75	6.67
	Prefix	45.00 (−7.2)	76.35 (−9.4)	3.33 (−3.3)
	Infix	49.40 (−2.8)	85.97 (+0.2)	**10.00** (+3.3)
	Suffix	49.40 (−2.8)	**86.73** (+1.0)	**10.00** (+3.3)
Qwen2.5-Math-7B-Instruct	Baseline	83.20	95.45	13.33
	Prefix	81.40 (−1.8)	94.92 (−0.5)	13.33 (+0.0)
	Infix	**84.20** (+1.0)	95.60 (+0.2)	**16.67** (+3.3)
	Suffix	83.40 (+0.2)	**95.68** (+0.2)	**16.67** (+3.3)
Qwen2.5-Math-1.5B-Instruct	Baseline	73.00	85.29	10.00
	Prefix	74.80 (+1.8)	85.29 (+0.0)	13.33 (+3.3)
	Infix	**75.20** (+2.2)	**85.75** (+0.5)	6.67 (−3.3)
	Suffix	74.20 (+1.2)	84.69 (−0.6)	**20.00** (+10.0)
Base Models (Raw Text)				
Qwen2.5-Math-7B	Baseline	**72.20**	84.15	6.67
	Prefix	71.60 (−0.6)	**84.31** (+0.2)	**16.67** (+10.0)
	Infix	63.20 (−9.0)	64.29 (−19.9)	**16.67** (+10.0)
	Suffix	55.60 (−16.6)	54.13 (−30.0)	13.33 (+6.7)
Qwen2.5-Math-1.5B	Baseline	61.40	**77.86**	**16.67**
	Prefix	**64.40** (+3.0)	74.30 (−3.6)	10.00 (−6.7)
	Infix	63.20 (+1.8)	73.92 (−3.9)	10.00 (−6.7)
	Suffix	54.60 (−6.8)	58.61 (−19.3)	3.33 (−13.3)
Base Models + ChatML (Format Mismatch)				
Qwen2.5-Math-7B	Baseline	52.20	58.30	**23.33**
	Prefix	59.20 (+7.0)	56.41 (−1.9)	3.33 (−20.0)
	Infix	62.00 (+9.8)	69.07 (+10.8)	**23.33** (+0.0)
	Suffix	**70.40** (+18.2)	**75.97** (+17.7)	**23.33** (+0.0)
Qwen2.5-Math-1.5B	Baseline	34.40	37.45	13.33
	Prefix	**63.40** (+29.0)	**67.02** (+29.6)	**20.00** (+6.7)
	Infix	39.20 (+4.8)	50.42 (+13.0)	6.67 (−6.7)
	Suffix	51.00 (+16.6)	50.64 (+13.2)	13.33 (+0.0)

Table 6: Number of RSP tokens (L) used for each model and benchmark.

Model	MATH-500	GSM8K	AIME24
LLaMA-3.1-8B-Instruct	15	20	15
Qwen2.5-Math-7B-Instruct	20	20	20
Qwen2.5-Math-1.5B-Instruct	20	10	20
Qwen2.5-Math-7B	10	10	20
Qwen2.5-Math-1.5B	10	10	20
Qwen2.5-Math-1.5B (ChatML)	20	20	10
Qwen2.5-Math-7B (ChatML)	20	20	20

A.1 RSP Injection Positions and Prompt Templates

Below we show the prompt structure for each injection position across all model types. Blue text indicates RSP embeddings, and purple text indicates special tokens.

A.1.1 Prefix

RSP embeddings are concatenated before the prompt embeddings. No text modification is applied.

ChatML (Qwen Instruct / Base + ChatML).

```
[Random Embeddings (L tokens)]
<|im_start|>system
Please reason step by step, and put your final answer within \boxed{}.<|im_end|>
<|im_start|>user
{question}<|im_end|>
<|im_start|>assistant
```

LLaMA Chat Template.

```
[Random Embeddings (L tokens)]
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>
Cutting Knowledge Date:  December 2023
Today Date:  26 Jul 2024
Please reason step by step, and put your final answer within \boxed{}.<|eot_id|>
<|start_header_id|>user<|end_header_id|>
{question}<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Raw Text (Qwen Base).

```
[Random Embeddings (L tokens)]
{question}
Please reason step by step, and put your final answer within \boxed{}.
```

A.1.2 Infix

A description text and special tokens are inserted into the prompt. The special token positions are then replaced with RSP embeddings at the embedding level.

ChatML (Qwen Instruct / Base + ChatML).

```
<|im_start|>system
Please reason step by step, and put your final answer within \boxed{}.<|im_end|>
<|im_start|>user
{question}
There are {L} special tokens that contain compressed latent reasoning information that
might be useful for your reasoning.  If these tokens are useful for your case, you can
use them as reference.  If these tokens are not useful for your case, you can ignore
them and focus back to solving the problem.
Here are the {L} special tokens:  <special_token_1>...<special_token_L>
<|im_end|>
<|im_start|>assistant
```

LLaMA Chat Template.

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>
Cutting Knowledge Date:  December 2023
Today Date:  26 Jul 2024
Please reason step by step, and put your final answer within \boxed{}.<|eot_id|>
<|start_header_id|>user<|end_header_id|>
```

```
{question}
There are {L} special tokens that contain compressed latent reasoning information that
might be useful for your reasoning. If these tokens are useful for your case, you can
use them as reference. If these tokens are not useful for your case, you can ignore
them and focus back to solving the problem.
Here are the {L} special tokens:
<reserved_special_token_0>...
<reserved_special_token_L>
<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Raw Text (Qwen Base).

```
{question}
There are {L} special tokens that contain compressed latent reasoning information that
might be useful for your reasoning. If these tokens are useful for your case, you can
use them as reference. If these tokens are not useful for your case, you can ignore
them and focus back to solving the problem.
Here are the {L} special tokens: <special_token_1>...<special_token_L>
Please reason step by step, and put your final answer within \boxed{}.
```

A.1.3 Suffix

For chat-templated models, RSP embeddings are inserted immediately before the end-of-turn token. For raw text models, RSP embeddings are concatenated at the end of the prompt.

ChatML (Qwen Instruct / Base + ChatML).

```
<|im_start|>system
Please reason step by step, and put your final answer within \boxed{}.<|im_end|>
<|im_start|>user
{question}[Random Embeddings (L tokens)]<|im_end|>
<|im_start|>assistant
```

LLaMA Chat Template.

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>
Cutting Knowledge Date: December 2023
Today Date: 26 Jul 2024
Please reason step by step, and put your final answer within \boxed{}.<|eot_id|>
<|start_header_id|>user<|end_header_id|>
{question}[Random Embeddings (L tokens)]<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Raw Text (Qwen Base).

```
{question}
Please reason step by step, and put your final answer within \boxed{}.[Random
Embeddings (L tokens)]
```

A.2 Answer Extraction and Grading

The final answer is extracted from the model output through a fallback chain. Each step is attempted in order; if extraction fails, the next step is tried.

Answer Extraction Fallback Chain

1. `"final answer is $...$. I hope"` pattern (Minerva format)
2. `\boxed{...}`: last non-empty box is used; empty `\boxed{}` are skipped
3. Text following `"the answer is"`
4. Text following `"final answer is"`
5. Last number in the output (regex fallback)

Extracted answers are normalized by removing `$`, `\left`, `\right`, and unit strings. Grading is performed via symbolic equivalence checking: exact string match, numerical comparison (`math.isclose`, `rel_tol=1e-4`), and SymPy symbolic simplification.

A.3 Reproduced Prior Work Hyperparameters

All prior methods are reproduced on Qwen2.5-Math-1.5B-Instruct and Qwen2.5-Math-7B-Instruct, evaluated with greedy decoding (temperature $= 0$) and our unified answer extraction pipeline.

TTSV. Prefix length 20, trained for 20 epochs with AdamW optimizer, batch size 2, gradient accumulation 8 steps, and linear learning rate schedule. Learning rate is 1e-3 for Qwen-1.5B and 1e-5 for Qwen-7B.

LTPO. Table 7 reports the per-benchmark hyperparameters.

Table 7: LTPO hyperparameters.

Parameter	GSM8K	MATH-500	AIME24
tokens	8	8	8
steps	20	20	20
top_k	10	10	10
sigma	20	20	5
sigma_decay	0.95	0.95	0.95
lr	1e-2	5e-3	5e-2

SoftCoT. Projection module trained for 10 epochs. Thought tokens: 32 during training, 4 during evaluation. Batch size 4 for GSM8K, 1 for MATH with gradient accumulation 4. The small (assistant) model is Qwen2.5-Math-1.5B-Instruct in both configurations, while the large model is Qwen2.5-Math-1.5B-Instruct (learning rate 2e-5) or Qwen2.5-Math-7B-Instruct (learning rate 5e-6). For MATH-500 and AIME24 evaluation, we use the projection module trained on GSM8K, as it yields higher accuracy than the one trained on MATH.

A.4 Full Entropy Curves

Figure 4 shows the full normalized (0–100%) generation trajectories for entropy, top-1 probability, and varentropy, and Table 8 reports the corresponding scalar statistics including overall means, first-5% means, and the average number of generated tokens per problem. All methods converge to similar levels after the initial generation stage, confirming that the distributional changes induced by RSP are localized to the early reasoning phase.

Table 8: Per-step statistics for Qwen2.5-Math-1.5B-Instruct on MATH-500 across the full generation and the first 5% of generation steps, together with the average number of generated tokens per problem. Top-1 Prob (overall) is reported as a weighted average over the first-10%/middle/last-10% segment means with weights $0.1/0.8/0.1$. Extension of Figure 2 (main text) and Figure 4.

Metric	**Baseline**	**Prefix**	**Infix**	**Suffix**	**LTPO**	**SoftCoT**	**TTSV**
Tokens (mean)	575.7	558.4	549.9	573.3	592.3	594.4	577.4
Entropy (first 5%)	0.1332	0.1366	0.1402	0.1941	0.1662	0.4218	0.1359
Entropy (overall)	0.0871	0.0881	0.0899	0.1049	0.0909	0.1059	0.0864
Top-1 Prob (first 5%)	0.9535	0.9523	0.9525	0.9373	0.9437	0.9156	0.9534
Top-1 Prob (overall)	0.9684	0.9681	0.9678	0.9647	0.9683	0.9651	0.9685
Varentropy (first 5%)	0.2181	0.2207	0.2463	0.4010	0.2953	2.2234	0.2174
Varentropy (overall)	0.1353	0.1375	0.1422	0.2038	0.1452	0.2404	0.1361

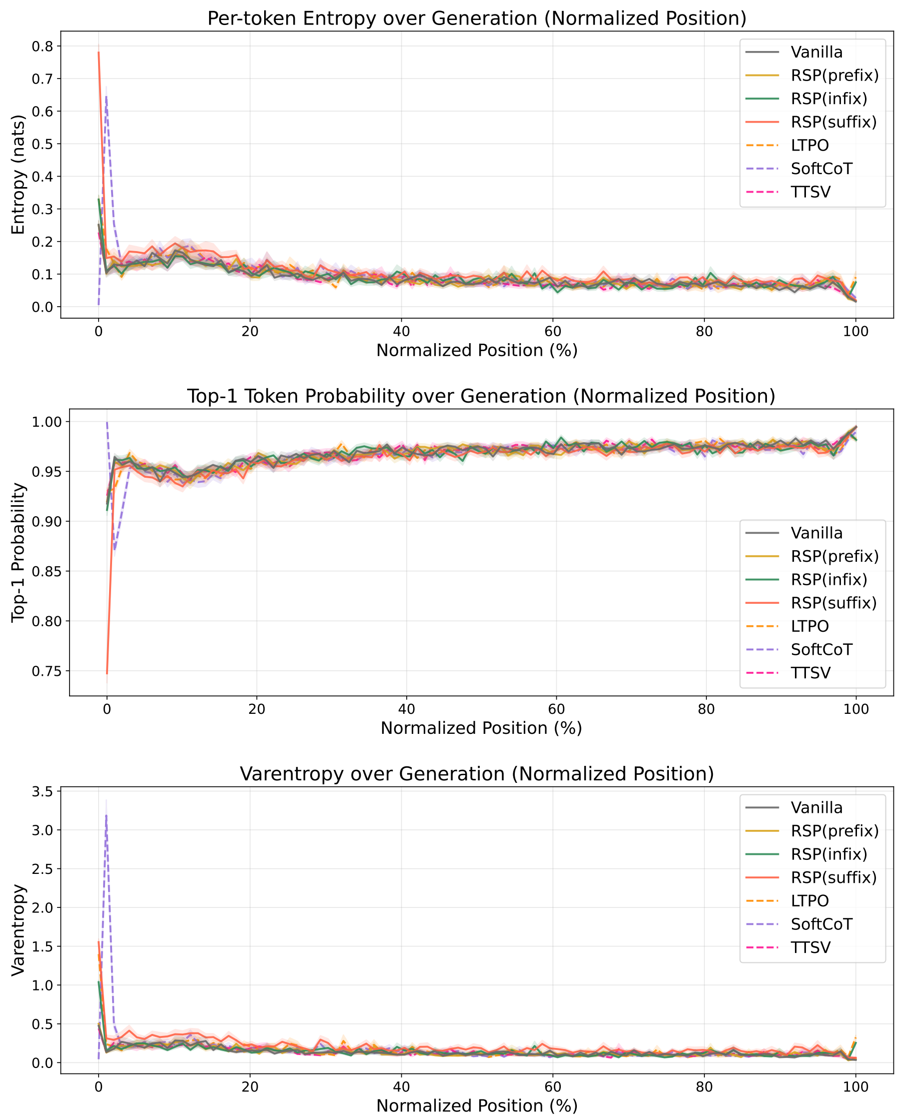


Figure 4: Mean entropy (top), top-1 probability (middle), and varentropy (bottom) over the full normalized generation trajectory (0–100%). Averaged over MATH-500, shading indicates ± 1 SEM. Solid lines: RSP variants and the baseline. Dashed lines: existing soft prompt methods. All methods converge after the initial stage.

B Token-Level Distribution Metrics

This appendix provides formal definitions for the three metrics used in the Token-level analysis of Section 4.4. Let P_{base} denote the baseline next-token distribution at $\tau = 1.0$ and P_{target} the candidate distribution being compared (e.g., the baseline at $\tau = 2.0$ or RSP at $\tau = 1.0$). All metrics are computed analytically from saved logits at the first generation step.

Spearman rank correlation. Spearman's ρ is the Pearson correlation coefficient between the token ranks of two distributions:

$$\rho \;=\; \mathrm{Pearson}(\mathrm{rank}(P_{\text{target}}),\; \mathrm{rank}(P_{\text{base}}))\,. \tag{3}$$

Because temperature scaling $P^{(\tau)} \propto P^{1/\tau}$ is a strictly monotone transform, the rank order of tokens is preserved exactly, so $\rho = 1$ for any $\tau > 0$ as a mathematical identity. Any value $\rho < 1$ therefore indicates rank reorganization beyond what temperature scaling can produce.

Mass outside top-K. Let $\mathrm{TopK}(P_{\text{base}})$ be the set of K tokens with the highest probability under P_{base}. The probability mass that the candidate distribution places *outside* this set is

$$\mathrm{MassOutside}_K \;=\; 1 - \sum_{t \in \mathrm{TopK}(P_{\text{base}})} P_{\text{target}}(t). \tag{4}$$

This directly measures support expansion: how much probability the candidate assigns to tokens that are not in the baseline's preferred set. Reported values use $K = 10$.

Jensen–Shannon divergence. The symmetrized, bounded version of Kullback–Leibler divergence:

$$\mathrm{JS}(P, Q) \;=\; \tfrac{1}{2}\,\mathrm{KL}(P \,\|\, M) + \tfrac{1}{2}\,\mathrm{KL}(Q \,\|\, M), \qquad M = \tfrac{1}{2}(P + Q). \tag{5}$$

We use base-2 logarithms so that JS $\in [0, 1]$. Tokens absent from the stored top-100 are assigned a sentinel logit (-10^9) before softmax, which is negligible for our setting since the top-5 covers more than 99.9% of the probability mass.

C Prompt-Law Properties Used by the Theory

This appendix isolates the only properties of the RSP law used in the main text: centeredness, direction-agnostic covariance under a variance budget, and positive probability on every nonempty open set. We avoid stronger semantic claims because Section 3.4 needs only these geometric and measure-theoretic facts.

C.1 Centered Perturbations Do Not Impose Systematic First-Order Drift

Let Δ_{ij} be the baseline logit gap between tokens i and j, and write the first-order surrogate as

$$\Delta_{ij}(\bar{h}) = \Delta_{ij} + b_{ij}^\top \bar{h}.$$

Proposition 3 (No systematic first-order drift). *If $\mathbb{E}[\bar{h}] = 0$, then*

$$\mathbb{E}\big[\Delta_{ij}(\bar{h})\big] = \Delta_{ij}$$

for every token pair (i, j). If a deterministic prompt h_0 is reused across runs, then

$$\Delta_{ij}(h_0) - \Delta_{ij} = b_{ij}^\top h_0$$

is a fixed directional bias.

Proof. The centered case is immediate from linearity of expectation:

$$\mathbb{E}\big[\Delta_{ij}(\bar{h})\big] = \Delta_{ij} + b_{ij}^\top \mathbb{E}[\bar{h}] = \Delta_{ij}.$$

The deterministic case follows by substitution. □

This proposition rules out a run-averaged first-order bias. It does not say that individual prompt draws leave the logits unchanged.

C.2 Proof of Theorem 2

For a unit vector b, the first-order perturbation energy available along b is $\mathrm{Var}_{h\sim D}(b^\top h) = b^\top \Sigma_D b$. The worst-case directional energy is therefore the minimum Rayleigh quotient of Σ_D.

Proof. For any symmetric covariance matrix Σ_D,

$$\min_{\|b\|=1} b^\top \Sigma_D b = \lambda_{\min}(\Sigma_D).$$

Let the eigenvalues of Σ_D be $\lambda_1, \ldots, \lambda_d$. Then

$$\lambda_{\min}(\Sigma_D) \le \frac{1}{d}\sum_{m=1}^{d} \lambda_m = \frac{\mathrm{tr}(\Sigma_D)}{d} \le \frac{\rho^2}{d}.$$

Equality holds if and only if all eigenvalues are equal, i.e., $\Sigma_D = (\rho^2/d)\mathbf{I}_d$. □

The proof uses only covariance. We choose a Gaussian law for RSP because it adds the open-set reachability property proved next.

C.3 Open-Set Reachability of Isotropic Gaussian Prompts

Proposition 4 (Open-set reachability). *Let $\bar{h} \sim \mathcal{N}(0, \sigma^2 \mathbf{I}_d)$ with $\sigma > 0$. Then every nonempty open set $U \subseteq \mathbb{R}^d$ satisfies*

$$\Pr(\bar{h} \in U) > 0.$$

Proof. The Gaussian density

$$(2\pi\sigma^2)^{-d/2} \exp\left(-\frac{\|\bar{h}\|^2}{2\sigma^2}\right)$$

is strictly positive at every point of $\mathbb{R}^d$. Every nonempty open set contains a ball of positive Lebesgue measure, and integrating a strictly positive density over that ball gives positive probability. □

This is the only support property used in Proposition 1.

D Proofs for the Transformer-Level Mechanism

This appendix follows the same order as the main theory section: exploration, annealing, and performance gain. The prompt-law facts used before these proofs are collected in Appendix C.

D.1 Exploration: attention decomposition and perturbation size

Proposition 5 (Attention decomposition)**.** *Let $\alpha_{ij}^{(\ell)}$ be the attention weights for query position i at layer ℓ, with n real tokens and L random tokens. Define*

$$w_{r,i}^{(\ell)} := \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)}$$

and, for $j \in [n]$,

$$\tilde{\alpha}_{ij}^{(\ell)} := \frac{\alpha_{ij}^{(\ell)}}{1 - w_{r,i}^{(\ell)}}.$$

Then

$$\mathbf{x}_i^{(\ell+1)} = (1 - w_{r,i}^{(\ell)})\, \tilde{\mathbf{x}}_i^{(\ell+1)} + \boldsymbol{\eta}_i^{(\ell)},$$

where

$$\tilde{\mathbf{x}}_i^{(\ell+1)} := \sum_{j=1}^{n} \tilde{\alpha}_{ij}^{(\ell)} \mathbf{v}_j^{(\ell)} \qquad \text{and} \qquad \boldsymbol{\eta}_i^{(\ell)} := \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)} \mathbf{v}_{r_j}^{(\ell)}.$$

Proof. The attention output is

$$\mathbf{x}_i^{(\ell+1)} = \sum_{j=1}^{n} \alpha_{ij}^{(\ell)} \mathbf{v}_j^{(\ell)} + \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)} \mathbf{v}_{r_j}^{(\ell)}.$$

By definition, $\sum_{j=1}^{n} \alpha_{ij}^{(\ell)} = 1 - w_{r,i}^{(\ell)}$, so

$$\sum_{j=1}^{n} \alpha_{ij}^{(\ell)} \mathbf{v}_j^{(\ell)} = (1 - w_{r,i}^{(\ell)}) \sum_{j=1}^{n} \frac{\alpha_{ij}^{(\ell)}}{1 - w_{r,i}^{(\ell)}} \mathbf{v}_j^{(\ell)} = (1 - w_{r,i}^{(\ell)})\, \tilde{\mathbf{x}}_i^{(\ell+1)}.$$

Substituting this identity gives the decomposition. $\square$

Corollary 1 (Magnitude scales with random-token attention)**.** *For every realization of the random values,*

$$\|\boldsymbol{\eta}_i^{(\ell)}\| \le w_{r,i}^{(\ell)} \max_{j \in [L]} \|\mathbf{v}_{r_j}^{(\ell)}\|.$$

Proof. By the triangle inequality,

$$\|\boldsymbol{\eta}_i^{(\ell)}\| = \left\| \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)} \mathbf{v}_{r_j}^{(\ell)} \right\| \le \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)} \|\mathbf{v}_{r_j}^{(\ell)}\| \le w_{r,i}^{(\ell)} \max_{j \in [L]} \|\mathbf{v}_{r_j}^{(\ell)}\|.$$

$\square$

Corollary 1 is the only quantitative fact needed in the main text. No independence assumption between attention weights and random keys is required. Once $w_{r,i}^{(\ell)}$ is small, the random contribution must be small as well.

D.2 Corollaries of Theorem 1 on cache growth

The two envelopes yield the corollaries below.

Corollary 2 (Sequence-wise annealing of the fixed-gap envelope)**.** *For fixed L and $\Delta_i^{(\ell)}$, the upper bound*

$$f(n) = \frac{L}{L + n\exp(\Delta_i^{(\ell)}/\sqrt{d})}$$

is strictly decreasing in n. Cache growth alone therefore narrows the largest RSP attention mass compatible with that gap.

During autoregressive decoding $\Delta_i^{(\ell)}$ also moves because queries, keys, and hidden states co-evolve. The accurate reading is not "the realized mass decreases at every step" but "the envelope compatible with a fixed gap shrinks as the cache grows."

Proof. Differentiate $f(n)$ with respect to n:

$$f'(n) = -\frac{L\exp(\Delta_i^{(\ell)}/\sqrt{d})}{\left(L + n\exp(\Delta_i^{(\ell)}/\sqrt{d})\right)^2} < 0.$$

Thus $f(n)$ is strictly decreasing. $\square$

Corollary 3 (Vanishing random contribution under annealing)**.** *If $n \to \infty$ while L and $\Delta_i^{(\ell)}$ remain fixed, $w_{r,i}^{(\ell)} \to 0$, and for every realization of the random values*

$$\|\boldsymbol{\eta}_i^{(\ell)}\| \le w_{r,i}^{(\ell)} \max_{j\in[L]} \|\mathbf{v}_{r_j}^{(\ell)}\| \to 0.$$

Proof. $w_{r,i}^{(\ell)} \to 0$ by Corollary 2. Apply Corollary 1 to bound $\|\boldsymbol{\eta}_i^{(\ell)}\|$. The maximum norm is finite for any realization of finitely many sampled values. $\square$

D.3 Performance gain: from local admission to Pass@N

The main text §3.4 uses a first-order surrogate to state two claims: a baseline-excluded token can enter the local top-K set, and independent reruns can turn such local openings into Pass@N gains. The proofs below use only one support condition on the centered prompt law D:

$$\text{(S1)} \qquad D(U) > 0 \quad \text{for every nonempty open set } U \subseteq \mathbb{R}^d.$$

Proposition 4 shows that the isotropic Gaussian law of §3.4 satisfies (S1).

D.3.1 Autoregressive Formulation

In autoregressive decoding, the joint distribution over a sequence $y_{1:T}$ factorizes as

$$\Pr(y_{1:T} \mid x) = \prod_{t=1}^{T} p(y_t \mid x, y_{<t}),$$

where each step is governed by a prefix-conditioned distribution. Any perturbation introduced by RSP affects generation through a sequence of local changes to $p(y_t \mid x, y_{<t})$, rather than a single global distribution.

D.3.2 Local Linearization of Logits under RSP

Throughout this subsection, $\bar{h} \in \mathbb{R}^d$ denotes *one* centered prompt vector from the RSP definition (Eq. (1) in §3.1), treated as a per-token surrogate. The actual implementation uses a length-L prompt $\mathbf{H} = (h_1, \ldots, h_L)$ with L independent draws; the arguments below isolate how one local perturbation can shift the logits. We model the perturbed logits as a function $z_i(\bar{h})$, assuming local smoothness in a neighborhood of $\bar{h} = 0$:

$$z_i(\bar{h}) = z_i(0) + \nabla_h z_i(0)^\top \bar{h} + r_i(\bar{h}),$$

where $r_i(\bar{h}) = O(\|\bar{h}\|^2)$. Defining $a_i := \nabla_h z_i(0)$, we obtain the local affine surrogate

$$z_i^{\mathrm{lin}}(\bar{h}) := z_i + a_i^\top \bar{h}.$$

This approximation is used as a first-order surrogate for branch opening, not as an exact global model of the transformer logits.

D.3.3 Proof of Proposition 1

Let

$$\mathcal{G}_{i,K} \ := \ \big\{\bar{h} \in \mathbb{R}^d : \#\{j \neq i : z_j^{\mathrm{lin}}(\bar{h}) > z_i^{\mathrm{lin}}(\bar{h})\} \leq K - 1\big\}.$$

Proof. Assume $\mathcal{G}_{i,K}$ contains a nonempty open set U. By (S1), $D(U) > 0$. Since $U \subseteq \mathcal{G}_{i,K}$,

$$\Pr_{\bar{h}}(\bar{h} \in \mathcal{G}_{i,K}) \ \geq \ D(U) \ > \ 0.$$

This is exactly the event that token i enters the surrogate top-K set.

A useful sufficient condition is the existence of some $\bar{h}_0$ at which token i strictly outranks all but at most $K-1$ other tokens in the affine surrogate. By continuity, a neighborhood of $\bar{h}_0$ is then contained in $\mathcal{G}_{i,K}$. □

D.3.4 Proof of Proposition 2

This proposition does not derive $p_{\min}(x) > 0$ from the mechanism. It only converts a task-side lower bound and independence into Pass@N growth.

Proof. Let A_n be the event that the n-th independent RSP run reaches a correct trajectory on problem x, and write $p_n(x) := \Pr(A_n) \geq p_{\min}(x)$ as in (M1). Under (M2),

$$\Pr\Big(\bigcap_{n=1}^{N} A_n^c\Big) = \prod_{n=1}^{N}(1 - p_n(x)) \leq \prod_{n=1}^{N}(1 - p_{\min}(x)) = (1 - p_{\min}(x))^N.$$

Taking complements gives

$$\Pr\big(\text{Pass@}N \text{ on } x\big) = \Pr\Big(\bigcup_{n=1}^{N} A_n\Big) \geq 1 - (1 - p_{\min}(x))^N.$$

The bound $1 - x \leq e^{-x}$ for $x \in [0,1]$ then yields

$$1 - (1 - p_{\min}(x))^N \ \geq \ 1 - e^{-N p_{\min}(x)}.$$

□

E Attention Mass Analysis

This appendix formalizes the per-token attention mass reported in Figure 1 and the exclusion criteria applied to the reasoning span and the layer axis. For each of the 500 MATH-500 problems, a fresh RSP $\mathbf{H} \in \mathbb{R}^{L \times d}$ with $L = 10$ is sampled independently, so the reported heatmaps average over both problem variability and RSP-vector variability.

E.1 Per-Token Attention Mass

For each problem, we measure attention on the concatenated sequence [prompt; $\mathbf{H}$; generation], where the generation is the trajectory produced by the same model under matching RSP injection. Let $\alpha_{i,j}^{(\ell,h)}$ denote the post-softmax attention weight at layer ℓ, head h, from query position i to key position j, satisfying $\sum_j \alpha_{i,j}^{(\ell,h)} = 1$ under the causal mask. Let Q and R denote the index sets of question and RSP tokens with sizes $|Q|$ and $|R| = L$, and let H be the number of heads. The per-token attention mass that query i directs to each group at layer ℓ is

$$\mathrm{PTM}_Q[\ell, i] = \frac{1}{|Q|H} \sum_{h=1}^{H} \sum_{j \in Q} \alpha_{i,j}^{(\ell,h)}, \qquad \mathrm{PTM}_R[\ell, i] = \frac{1}{|R|H} \sum_{h=1}^{H} \sum_{j \in R} \alpha_{i,j}^{(\ell,h)}. \tag{6}$$

Normalization by $|Q|$ and $|R|$ controls for group size, so both quantities express how much attention a single question (resp. RSP) token receives on average under the same softmax denominator. Question tokens thereby serve as a size-matched baseline that absorbs sequence-length effects common to both groups.

E.2 Heatmap Aggregation

We partition the layer indices and the reasoning position indices each into five equal-size bins $\{B_L^{(b)}\}_{b=1}^5$ and $\{B_P^{(b)}\}_{b=1}^5$. For sample s and target group $\bullet \in \{Q, R\}$, the value in cell (b_L, b_P) is

$$M_\bullet^{(s)}[b_L, b_P] = \frac{1}{|B_L^{(b_L)}| \cdot |B_P^{(b_P)}|} \sum_{\ell \in B_L^{(b_L)}} \sum_{i \in B_P^{(b_P)}} \mathrm{PTM}_\bullet[\ell, i]. \tag{7}$$

Figure 1 reports the sample average of Eq. (7) over the 500 valid samples.

E.3 Excluding the `\boxed{...}` Span

The reasoning position range terminates strictly before the final `\boxed{...}` span. Chain-of-thought outputs decompose into a *text* (content) component and a *pattern* (template) component that contribute independently to downstream performance [Madaan and Yazdanbakhsh, 2022]. The `\boxed{...}` wrapper is a canonical pattern: a short, stereotyped span whose role is to complete the answer format rather than to extend reasoning. Including it would therefore inject a template-driven attention signature unrelated to reasoning dynamics.

E.4 Excluding the Final Two Layers

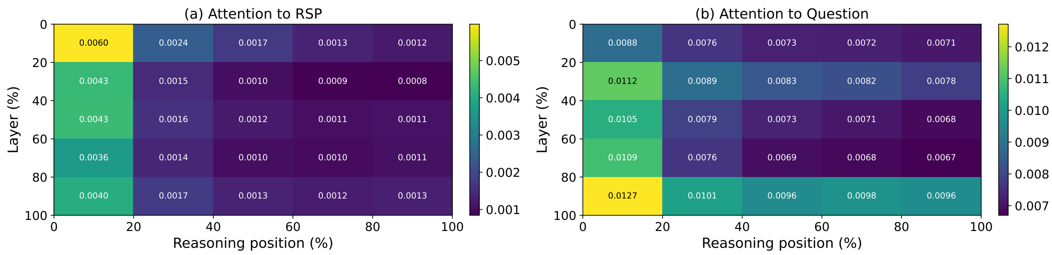
The layer axis further excludes the last two transformer layers. This choice is motivated by a standard late-layer effect rather than by any model-specific tuning decision.

Output-projection specialization. Late-layer hidden states lie in an approximately linear relationship with vocabulary logits and therefore function as a progressive linear readout rather than as representation-transforming blocks [Belrose et al., 2023]. Consistently, late-layer feed-forward modules have been characterized as output-distribution shaping mechanisms [Geva et al., 2021]. Attention in these layers therefore reflects output formation rather than reasoning computation.

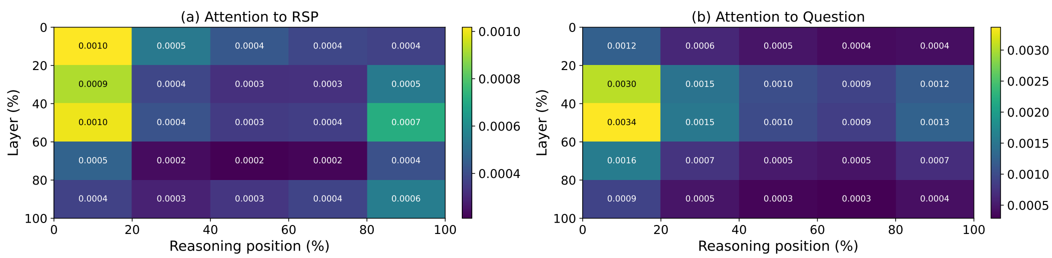
Excluding the final two layers keeps the heatmap focused on reasoning-phase dynamics instead of readout-phase dynamics. This exclusion is not load-bearing for the main claim: the sequence-direction attenuation emphasized in the main text is also visible without it, and Theorem 1 concerns sequence-direction attenuation rather than layer-wise monotonicity.

E.5 Additional Suffix Heatmaps

Figure 5 reports the same per-token RSP attention analysis under suffix injection for Qwen2.5-Math-1.5B-Instruct and LLaMA-3.1-8B-Instruct. For the late-layer reasons discussed above, the pattern does not generalize uniformly across every cell; nevertheless, the overall trend is consistent across both models: question-token attention varies broadly across layers, whereas RSP attention shows the layer-wise and sequence-wise decay predicted by Theorem 1.



(a) Qwen2.5-Math-1.5B-Instruct, suffix



(b) LLaMA-3.1-8B-Instruct, suffix

Figure 5: Per-token RSP attention mass under suffix injection for the remaining two models (500 MATH-500 samples each). Axes and preprocessing match Figure 1: x-axis is reasoning position binned into five quantiles, y-axis is layer depth binned into five quantiles (top = shallow), color encodes the 500-sample mean per-token attention assigned to RSP tokens, and tokens inside `\boxed{}` spans and the last two layers are excluded.

F GRPO Formulation

For a given prompt q, the behavior policy $\pi_{\theta_{\text{old}}}$ samples a group of G responses $\{o_i\}_{i=1}^{G}$ with corresponding rewards $\{R_i\}_{i=1}^{G}$. The advantage of each response is computed by normalizing the group-level rewards:

$$\hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^{G})}{\text{std}(\{R_i\}_{i=1}^{G})}. \tag{8}$$

The policy is updated via a clipped objective with a KL penalty:

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\Big(\min\Big(r_{i,t}\hat{A}_{i,t},\ \text{clip}(r_{i,t}, 1-\varepsilon, 1+\varepsilon)\,\hat{A}_{i,t}\Big) - \beta D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})\Big)\right], \tag{9}$$

where $r_{i,t} = \pi_\theta(o_{i,t} \mid q, o_{i,<t})/\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$ is the importance sampling ratio, ε is the clipping range, and β controls the strength of the KL penalty against the reference policy π_{ref}.

G Broader Impacts

This work provides empirical and theoretical analysis of how random embedding injection affects the reasoning behavior of large language models. The contributions are primarily foundational, with potential downstream implications for reinforcement learning-based post-training of reasoning models such as GRPO.

Positive impacts. RSP-induced trajectory diversity could improve the sample efficiency of RL-based LLM post-training by providing richer learning signals within group-relative advantage estimation. This has the potential to reduce compute costs associated with alignment and reasoning-oriented fine-tuning, making such methods more accessible to research groups with limited resources.

Potential concerns. Methods that expand the effective output support of LLMs may increase the reachability of low-probability tokens, which includes both desirable alternative reasoning paths and potentially unsafe or off-distribution outputs. Practitioners applying RSP in production systems should combine it with existing safety filtering and alignment mechanisms rather than treating it as a standalone diversity source.

Limitations on impact scope. Since this work focuses on analysis rather than deploying a new model or dataset, the direct societal impact is limited. The broader implications described above are contingent on future work integrating RSP into applied training pipelines.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: **[TODO]**

   Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
    (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
    (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
    (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
    (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: **[TODO]**

   Justification: **[TODO]**

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: **[TODO]**

    Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: **[TODO]**

    Justification: **[TODO]**

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.